Phishing URL Detector

William Waly, Fahad Ahmed, Patrik Thormodsen

1. Introduction

The goal of this project was to develop a small website where users can enter a URL and receive a machine learning based prediction of whether it is phishing or legitimate. The product is intended for people who want help assessing unfamiliar or suspicious looking links before opening them. Machine learning is a promising approach because it can learn combinations of URL characteristics from labeled examples, rather than relying only on manually defined rules. Without machine learning, users could inspect the domain and URL structure themselves or check the link against known phishing links. Our system analyzes characteristics extracted from the URL text, without visiting the website. Its prediction should only be used as an additional warning signal and not a guarantee that the website is safe or unsafe.

We evaluated the model using accuracy, precision, recall and F1-score, treating phishing as the positive class. Accuracy measures the proportion of correct predictions overall. Precision measures how many URLs flagged as phishing are actually phishing, while recall measures how many phishing URLs the model detects. Recall is particularly important because missed phishing URLs could give users false reassurance. However, precision also matters, since frequent false alarms could reduce trust in the system. F1-score balances precision and recall, and a confusion matrix shows the different types of prediction errors.

We compared the model against a baseline that always predicts the most common class. The prototype was considered successful if it outperformed the baseline on unseen evaluation data, detected most phishing examples and provided a clear result that communicated its limitations.

2. Data and Preprocessing

We initially used the PhiUSIIL Phishing URL dataset from the UCI Machine Learning Repository. It contains 235,795 examples and 56 columns, including URLs, extracted characteristics and classification labels. There are 134,850 legitimate examples and 100,945 phishing examples. The labels are encoded as 1 for legitimate and 0 for phishing. These labels provide the ground truth for supervised learning. However, we rely on the labeling process used by the dataset creators and cannot independently verify every example.

Our initial exploration found no missing values. We started by training models using numerical features extracted from URL text, such as URL length, domain length and counts of digits and special characters. Features requiring webpage content were excluded because our application does not visit websites.

After the initial experiments, we switched to the PhreshPhish dataset, which contains more recent phishing and legitimate URLs collected during 2024–2025. It also provides a separate test split collected later than its training data. This allowed us to evaluate how well our model performs on newer URLs.

For our final model, we used the URL text directly instead of manually extracted numerical features. Before training, we remove the scheme and `www.`, convert the hostname to lowercase and keep the path, query and other URL components. We then use TF-IDF with character n-grams of length 3–5 to convert the text into numerical features. This allows the model to recognize patterns in short sequences of characters, such as parts of domains and URL paths. The same preprocessing is used during training and prediction to ensure consistent inputs.

During model development, we separated the data into training, validation and holdout sets. We made sure that URLs from the same website did not appear in more than one split, reducing the risk of data leakage and overly optimistic results. The final test set comes from PhreshPhish's official test split, where 28% of the URLs share websites with the training data. We therefore also evaluated the model on URLs from previously unseen websites, where it achieved 97.9% precision and 80.1% recall, close to the overall results.

Performance on these datasets may not fully reflect performance on future phishing campaigns or URLs collected from other sources.

URLs can contain personal information or access tokens. Our application therefore avoids deliberately storing submitted URLs, and users should avoid submitting private links. Predictions are presented as estimates rather than guarantees of safety. We also credit the dataset creators and comply with their CC BY 4.0 licenses.

3. Modeling

We explored several machine learning models throughout the project. Our first experiments used a random forest trained on numerical features from the PhiUSIIL dataset. After switching to PhreshPhish, we compared decision trees, random forests and gradient boosting using structural URL features. We also trained a text-based model using TF-IDF with SGDClassifier and logistic loss.

In addition, we experimented with combining text and structural features, as well as ensemble methods such as stacking.

Model comparison

The table below shows selected results from our development benchmark of 10,000 URLs from PhreshPhish. Recall was measured using a threshold selected to produce approximately 1% false positives on the validation set.

Model	PR-AUC	Phishing recall
Decision Tree	0.912	0.445
Random Forest	0.932	0.501
Gradient Boosting	0.934	0.536
TF-IDF + SGDClassifier	0.977	0.726
TF-IDF + structural features	0.985	0.830
Stacking	0.987	0.840

The text-based models performed better than models using only structural features. Adding structural features increased recall, but also doubled the false alarm rate on legitimate homepages from 3.3% to 6.6%. This did not meet the acceptance criteria we had set for the experiments.

Stacking achieved the best results on several metrics, but required two models and a more complicated prediction pipeline. We therefore selected TF-IDF with SGDClassifier because it provided a good balance between performance and simplicity.

Final evaluation

After selecting the model, we evaluated it on a reserved test set containing 10,000 URLs, using a prediction threshold of 0.5.

Metric	Result
Accuracy	90.6%
Precision (phishing)	97.7%
Recall (phishing)	80.8%
F1-score	88.5%
False positive rate	1.5%
ROC-AUC	0.979
PR-AUC	0.977

For comparison, a baseline that always predicts the majority class would achieve 55.2% accuracy on the same test set, but would detect no phishing URLs.

The confusion matrix below shows the number of correct and incorrect predictions.

	Predicted phishing	Predicted legitimate
Actual phishing	3,618	858
Actual legitimate	85	5,439

The model correctly identified 3,618 out of 4,476 phishing URLs, meaning it detected approximately four out of five phishing examples. It also incorrectly classified 85 legitimate URLs as phishing. These results show that the model performs significantly better than the majority-class baseline, although it still makes mistakes.

Feature importance and prediction errors

We also investigated feature importance in the random forest. The most important structural features were the number of slashes in the URL path, path length and hostname length. These features suggest that URL structure can help distinguish phishing from legitimate URLs, although the text-based model performed better overall.

Our error analysis showed that some phishing URLs are difficult to detect because they are hosted on legitimate platforms such as Google Sites or Webflow. These URLs can look similar to normal links. The model also sometimes incorrectly flags legitimate homepages and does not reliably detect look-alike domains such as `netf1ix.com`.

These limitations are important because an incorrect prediction could either create unnecessary concern or give users false reassurance. The website therefore makes it clear that its predictions are not guarantees of safety.

4. Deployment

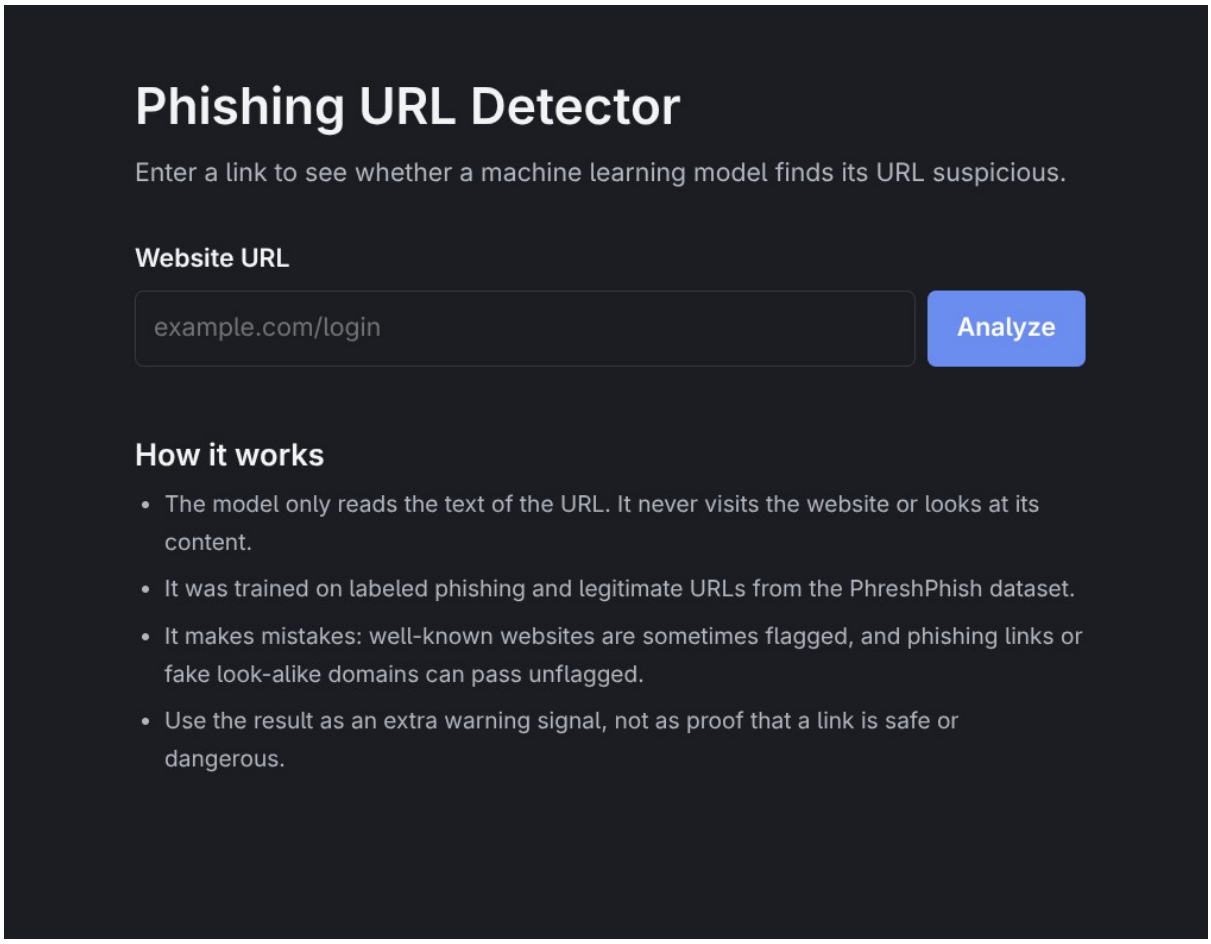
The trained model is deployed as part of a web application. We developed the frontend using React and the backend using FastAPI. The model is saved using joblib and loaded by the backend when the application starts.

When a user submits a URL, the frontend sends it to the backend through a prediction endpoint. The backend applies the same preprocessing used during training and passes the result to the model. The prediction is then returned to the frontend, which displays whether the URL was flagged as potential phishing.

The website also explains that the model only analyzes URL text and that an unflagged URL is not necessarily safe. We do not display the raw model score as a probability, since it has not been calibrated.

The application runs in Docker containers, with Nginx serving the frontend. It is hosted on our own server and made publicly accessible through a Cloudflare Tunnel.

Website: https://phishing.patreek.no



The project met all three success criteria: the model outperformed the majority-class baseline, detected 80.8% of phishing URLs, and the website clearly communicates the limitations of its predictions.

Monitoring and future improvements

Phishing techniques change over time, meaning the model may become less accurate as new types of phishing URLs appear. To maintain its performance, we would need to retrain it using newer data and evaluate updated models before deploying them.

We could also monitor the proportion of submitted URLs flagged as phishing over time, without storing the actual URLs. Significant changes could indicate that the types of URLs being submitted have changed. We use pinned library versions to ensure compatibility with the saved model.

Possible improvements include better coverage of legitimate homepages in the training data, improved detection of look-alike domains and additional evaluation using recently collected phishing URLs.

5. References

1. Dalton et al. (2025). PhreshPhish: A Real-World, High-Quality, Large-Scale Phishing Website Dataset and Benchmark. arXiv:2507.10854. Dataset available on Hugging Face (phreshphish/phreshphish), CC BY 4.0.
2. Prasad and Chandra (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. Computers & Security, 136. Dataset available from the UCI Machine Learning Repository, CC BY 4.0.
3. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12.
4. Official documentation for Scikit-learn, FastAPI, React, Docker, Nginx and Cloudflare Tunnel.

AI assistance: Claude Code and ChatGPT were used to assist with parts of the code, experiments, evaluation, web application, deployment configuration, and report writing. The project group reviewed the generated content, verified the results, and made the final decisions.